fine-tuning recipes for supported hardware. Use its current compatibility instructions rather than copying an old installation command. The objective is to learn controlled adaptation, not win a dependency-management contest.

For readers without a dedicated GPU, renting a compatible accelerator for a controlled experiment is another possibility, but uploading private training data to a third-party service requires explicit authorisation. Check session-storage behaviour, regional data requirements and total costs before making that choice. Public demonstration datasets are safer for learning the mechanics.

Read the curves, not the hype

During training, TRL reports loss and can evaluate against your validation set. Falling training loss indicates that the model is becoming better at predicting target tokens. It does not prove that answers are useful. If training loss improves while validation results deteriorate, overfitting is a plausible explanation. Stop, inspect examples and consider fewer epochs or better data diversity.

Also examine the generated outputs. Is the model returning the expected field names? Does it handle an entirely new complaint? Does it admit uncertainty instead of inventing warranty entitlements? Compare multiple random seeds where practical, especially when your validation set is small.

Maintain a simple experiment log: base-model revision, dataset version, adapter configuration, training settings, hardware, training duration and held-out scores. Record failure examples as carefully as successes. In a business application, a model that produces valid JSON 98 times out of 100 may still need schema validation and error handling, especially if the failures trigger real actions.

Save, load and reuse adapters

A successful LoRA run produces portable adapter files, not a complete independent model. To test one later, load the same base checkpoint and attach the adapter with PEFT's `PeftModel.from_pretrained(base, "models/support-lora")`. Use the base tokenizer's chat template for your prompts and put the model into evaluation mode.

Keep adapters separate when you want to switch tasks, compare revisions or avoid duplicating the underlying weights. You might use one adapter for support classification and another for extracting product specifications, but each still needs its own tests. PEFT allows multiple adapters to be installed and selected by name; an adapter is not a plug-in that magically works with unrelated model families.

For some inference workflows, `merge_and_unload()` folds a compatible LoRA adapter into the base weights and produces an ordinary merged checkpoint. That can simplify serving but increases storage, may remove easy switching and requires attention to precision and compatibility. Do not merge casually into four-bit weights and expect a lossless result. Keep the original adapter, base reference, licence details and benchmark results whether you merge or not.

WHY A LOW TRAINING LOSS CAN MISLEAD

Imagine training on a hundred near-identical replies: every example begins "Thank you for contacting support" and ends with the same template. Loss may drop impressively because predicting repetitive language is easy. Yet the model could still classify cracked screens as return requests or invent a free replacement. The experiment looks successful only because its score rewards imitation, not business correctness.

Prevent this by varying cases, checking labels, separating related examples across dataset splits and scoring the actual fields you care about. Test inputs deliberately missing critical information. Check behaviour in Hindi and Hinglish if these languages are part of your intended workload. Prefer a modest improvement that survives genuinely unseen requests over a spectacular training curve that proves memorisation. Training metrics are diagnostics; application tests are the verdict.

What comes after supervised fine-tuning?

SFT teaches a model from examples of preferred responses. More advanced post-training can express preferences between alternatives. Direct Preference Optimization, or DPO, uses examples containing a prompt, a preferred answer and a rejected answer. For instance, one reply may acknowledge uncertainty while another invents a warranty condition. DPO can favour the better response, provided the preference labels are trustworthy.

Other methods, including reward-based approaches such as GRPO, are useful in specialised settings but add considerable design and evaluation complexity. Hugging Face TRL supports these workflows, yet they are not obligatory steps in a small-business AI project. Do not move to a more elaborate method simply because the acronym sounds newer.

Your milestone is concrete: a documented dataset, a reproducible adapter, an untouched test set and evidence of improvement over a well-prompted base model. Fine-tuning modifies behaviour; it does not replace retrieval, tool permissions or application testing. With that distinction in place, we can give the customised model the next capability it lacks: the power to act through tools.